UNIT 1 · TOPIC 1.1

Social Engineering

Assignment, Answer Key

Teacher answer key, 28 points total.

Practice Questions

1.1.a · Psychological tactics (intimidation, urgency, authority, scarcity)

1. An email says 'Your account closes in 1 hour unless you verify now.' Which tactic is this PRIMARILY using? (1 pt)

- ○ A. Scarcity
- ○ B. Reciprocity
- ○ C. Authority
- **● D. Urgency**

Why: Manufactured time pressure ('1 hour', 'now') is urgency; it pushes you to act before verifying. It is not scarcity, because nothing is described as limited in supply, and nobody claims special authority.

2. A message reads: 'This is the IRS. Pay the penalty within 24 hours or face arrest.' Select ALL tactics present. (3 pts)

- **■ A. Authority**
- **■ B. Urgency**
- **■ C. Intimidation**
- □ D. Scarcity

Why: IRS = authority, '24 hours' = urgency, 'penalty'/'arrest' = intimidation. No scarcity.

3. Why is recognizing the underlying TACTIC more useful than memorizing specific scam templates? (1 pt)

- ○ A. Scam templates are copyrighted, so security training cannot show real examples
- **● B. Tactics stay constant even when attackers change the brand, wording, or channel**
- ○ C. Spam filters already catch every known template, so only tactics are left to learn
- ○ D. Each tactic appears in only one kind of scam, so naming it identifies the scam

Why: Surface details (brand, wording, channel, even AI-written text) change constantly, but the psychological levers do not, so recognizing the lever generalizes to messages you have never seen. Filters do not catch every template, and tactics are not tied to one scam type: urgency shows up in bank, IRS and CEO scams alike.

4. Name the single best defensive response when you notice manufactured emotional pressure in a message. (1 pt)

Acceptable answers: slow down; verify; separate channel; verify through a trusted channel; stop and verify

Why: Slow down and verify the request through a separate, trusted channel.

1.1.b · Elicitation; malicious links and files

5. An attacker tells a receptionist 'I heard your VPN uses port 1433, or was it 443?' hoping the receptionist corrects them. Which elicitation gambit is this? (1 pt)

- **● A. Deliberate incorrect statement**
- ○ B. Flattery / appeal to expertise
- ○ C. Mutual disclosure

○ D. Third-party reference

Why: Stating something wrong to provoke a correction is the deliberate-incorrect-statement gambit. The target volunteers the accurate detail to fix the error.

6. Which of the following are malicious indicators in the URL http://paypa1.com/verify.exe ? Select ALL that apply. (2 pts)

■ **A. lookalike-domain (paypa1.com imitates paypal.com)**

□ B. mismatched-link (the link text differs from where it really goes)

■ **C. dangerous-extension (.exe)**

□ D. ip-url (the host is a numeric address)

Why: paypa1.com swaps the letter l for the digit 1, so it is a lookalike domain, and verify.exe downloads a program. There is no separate link text shown here to compare against, so mismatched-link cannot be judged, and the host is a name, not an IP address.

7. A filename shows as 'invoice.pdf' in Windows Explorer. When you look at the true name it is 'invoice.pdf.exe'. What technique does this use? (1 pt)

○ A. Typosquatting

● **B. Double extension**

○ C. Homograph attack

○ D. URL shortening

Why: A double extension exploits Windows hiding known extensions by default, making .exe appear as .pdf in the file browser.

8. Why does HTTPS in a URL NOT prove that a website is safe? (1 pt)

Acceptable answers: encrypted; encryption only; certificate; tls only; attacker can get certificate; certificate does not verify legitimacy

Why: HTTPS only proves the connection is encrypted. Attackers freely obtain TLS certificates for malicious domains; the padlock does not mean the site is legitimate.

9. Which action BEST defends against elicitation when someone you do not recognize asks detailed questions about your network? (1 pt)

○ A. Answer only general questions and refuse anything specific

○ B. Ask them a security question only a real employee would know

● **C. Verify their identity through official channels before sharing any operational details**

○ D. Share only details that are already on the company website

Why: Elicitation works by collecting small, harmless-seeming pieces, so "general" or "public" answers still help the attacker build a picture, and a quiz question can be researched or guessed. Verifying identity through an official channel (a directory number, the security team) before sharing anything is the reliable defense.

1.1.c · Email, text, and social media attacks

10. An attacker registers 'paypa1.com' (digit 1 replacing the letter l) and sends password-reset emails from that domain. Which technique does this PRIMARILY represent? (1 pt)

● **A. Typosquatting (lookalike domain)**

○ B. Spear phishing

○ C. Smishing

○ D. Watering-hole attack

Why: Registering a near-miss spelling of a trusted domain, such as a digit 1 for the letter l, is typosquatting. A homograph attack is different, it swaps in look-alike letters from another alphabet, such as a Cyrillic a for a Latin a.

11. A text message says: 'UPS: Your package is held. Reschedule at http://bit.ly/upsdel.' What makes SMS (smishing) especially effective for this type of attack? (1 pt)

○ A. SMS messages cannot be filtered by spam engines

● **B. Limited screen real estate hides full URLs; people trust texts on personal devices**

○ C. Phones display URLs in full without truncation

○ D. SMS is encrypted end-to-end so attackers can bypass firewalls

Why: Smishing exploits the fact that phone screens show truncated or shortened URLs and that people have higher baseline trust in SMS messages received on personal devices.

12. A social media DM says: 'Your account expires tonight. Verify at http://192.168.5.1/login to avoid losing access.' Select ALL phishing indicators present. (3 pts)

■ **A. Shortened link / bare IP address URL**

■ **B. Urgency (account expiring tonight)**

■ **C. Credential request (verify to avoid losing access)**

□ D. Legitimate organizational sender

Why: The bare IP address URL triggers the shortened-link indicator, 'expires tonight' is urgency, and 'verify to avoid losing access' is a credential request. A legitimate organization does not send you to a bare IP address.

13. What distinguishes spear phishing from generic phishing? (1 pt)

○ A. Spear phishing is sent by text message, while phishing is sent by email

○ B. Spear phishing always carries malware, while phishing only steals passwords

○ C. Spear phishing targets executives, while phishing targets ordinary employees

● **D. Spear phishing uses pre-researched details about a specific target to increase believability**

Why: The defining feature is research: spear phishing is tailored to one person or group using details like their name, role and recent activity. It can use any channel and any payload. Phishing aimed at executives specifically has its own name, whaling, which is a subtype of spear phishing.

14. Name the one defensive action that works across email, SMS, and social media phishing when you suspect a message is fraudulent. (1 pt)

Acceptable answers: verify through a separate channel; verify independently; contact the organization directly; call the organization; look up the real contact; use a trusted contact method; verify out of band

Why: Verify through an independent channel, look up the organization's real contact information separately and reach out that way, rather than using any link, number, or address in the suspicious message.

Coding Assignments (Python 3)

1.1.a · Psychological tactics (intimidation, urgency, authority, scarcity)

3 pts · auto-graded on the site

Coding problem 1.1.a: Detect psychological pressure tactics

Skill: Analyze Risk · **Maps to:** 1.1.A, 1.1.B · **Language:** Python 3 (pytest)

Problem

Build the detector a phishing-awareness tool would use. Given the raw text of a message, return which of the four psychological pressure tactics it employs:

- `"urgency"`: manufactured time pressure
- `"authority"`: borrowed legitimacy (boss, IT, government, bank)
- `"scarcity"`: limited availability / fear of missing out
- `"intimidation"`: threats / fear

A message may use zero, one, or several tactics.

Function contract

Implement `solve(text: str) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the detected tactic labels as a **sorted, de-duplicated** list. Matching is case-insensitive. Detection is by the indicator phrase sets defined in `interface.md`.

Input

A single string, the message body (and/or subject) to analyze.

Output

A sorted list of unique tactic labels from the set above (empty list if none).

Examples

```
"Your account will be locked in 24 hours. Act now."        -> ["intimidation", "urgency"]
"This is the IT department. Limited spots — last chance."  -> ["authority", "scarcity"]
"Pay the penalty or we will take legal action immediately" -> ["intimidation", "urgency"]
"Lunch is in the kitchen, help yourself."                  -> []
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List

INDICATORS = {
    "urgency": [
        "urgent", "act now", "immediately", "right now", "asap", "expires",
        "within 24 hours", "24 hours", "as soon as possible", "don't delay", "do not delay",
    ],
    "authority": [
        "it department", "ceo", "manager", "administrator", "irs", "bank", "police",
        "official", "help desk", "human resources", "hr department",
    ],
    "scarcity": [
        "limited", "only", "last chance", "running out", "few left", "exclusive",
        "while supplies last", "act fast",
```

```
    ],
    "intimidation": [
        "legal action", "account suspended", "account will be locked", "penalty", "arrested",
        "fired", "terminated", "lawsuit", "consequences", "reported",
    ],
}


def solve(text: str) -> List[str]:
    low = (text or "").lower()
    found = {tactic for tactic, phrases in INDICATORS.items()
             if any(p in low for p in phrases)}
    return sorted(found)
```

1.1.b · Elicitation; malicious links and files

3 pts · auto-graded on the site

Coding problem 1.1.b: Malicious link and file indicator detector

Skill: Analyze Risk · **Maps to:** 1.1.A, 1.1.C · **Language:** Python 3 (pytest)

Problem

Build the analyzer a phishing-awareness tool would use to flag suspicious URLs and filenames. Given a single string that is either a URL or a filename, return which malicious indicators it exhibits.

Four indicator categories are defined:

- `"lookalike-domain"`: the URL hostname resembles a known brand but is not the real domain (typosquatting rules defined in `interface.md`)
- `"ip-url"`: the URL uses a raw IPv4 address as its host instead of a domain name
- `"dangerous-extension"`: the filename (or URL path) ends in an executable or script extension
- `"mismatched-link"`: the URL contains a brand keyword in a path or subdomain while the actual registrable domain belongs to a different host (defined precisely in `interface.md`)

Function contract

Implement `solve(url_or_filename: str) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Return the detected indicator labels as a **sorted, de-duplicated** list.

Input

A single string, either a URL (starting with `http://` or `https://`) or a bare filename.

Output

A sorted list of zero or more indicator labels drawn from `{"dangerous-extension", "ip-url", "lookalike-domain", "mismatched-link"}`.

Examples

```
"http://paypa1.com/verify"                  -> ["lookalike-domain"]
"http://192.168.4.99/reset"                 -> ["ip-url"]
"invoice.pdf.exe"                           -> ["dangerous-extension"]
"http://bankofamerica.com.secure-login.xyz" -> ["mismatched-link"]
"http://shop.example/track/44821"           -> []
"http://dropbx.com/s/Q3.scr"               -> ["dangerous-extension", "lookalike-domain"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
import re
from typing import List
from urllib.parse import urlparse


LOOKALIKE_DOMAINS = {
    "paypa1.com", "arnazon.com", "dropbx.com", "g00gle.com", "faceb00k.com",
    "rn1crosoft.com", "rnicrosott.com", "rnicrosoft.com", "linkedln.com",
    "app1e.com", "twiitter.com", "lnstagram.com",
}

DANGEROUS_EXTENSIONS = {
    ".exe", ".scr", ".bat", ".cmd", ".ps1", ".vbs", ".vbe", ".js", ".jse",
    ".wsf", ".wsh", ".jar", ".pif", ".com", ".hta", ".msi", ".dll", ".cpl", ".reg",
}

TRUSTED_DOMAINS = {
    "paypal.com", "amazon.com", "dropbox.com", "google.com", "facebook.com",
    "microsoft.com", "linkedin.com", "apple.com", "twitter.com", "instagram.com",
    "bankofamerica.com", "chase.com", "wellsfargo.com", "company.example",
    "shop.example", "meridian-corp.example",
}

TRUSTED_BRANDS = [
    "paypal", "amazon", "dropbox", "google", "facebook", "microsoft", "linkedin",
    "apple", "twitter", "instagram", "bankofamerica", "chase", "wellsfargo",
]

_IP_RE = re.compile(r'^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}(:\d+)?$')


def _registrable_domain(hostname: str) -> str:
    parts = hostname.lower().split(".")
    if len(parts) >= 2:
        return ".".join(parts[-2:])
    return hostname.lower()


def _final_extension(s: str) -> str:
    segment = s.rstrip("/").split("/")[-1].split("?")[0].split("#")[0]
    dot_idx = segment.rfind(".")
    if dot_idx == -1:
        return ""
    return segment[dot_idx:].lower()


def solve(url_or_filename: str) -> List[str]:
    s = url_or_filename or ""
    found = set()
    is_url = s.lower().startswith("http://") or s.lower().startswith("https://")
```

```
    if is_url:
        parsed = urlparse(s)
        hostname = parsed.hostname or ""
        reg_domain = _registrable_domain(hostname)

        if _IP_RE.match(parsed.netloc.lower()):
            found.add("ip-url")

        if reg_domain in LOOKALIKE_DOMAINS:
            found.add("lookalike-domain")

        if reg_domain not in TRUSTED_DOMAINS:
            url_lower = s.lower()
            url_without_reg = url_lower.replace(reg_domain, "", 1)
            for brand in TRUSTED_BRANDS:
                if brand in url_without_reg:
                    found.add("mismatched-link")
                    break

        ext = _final_extension(parsed.path)
        if ext in DANGEROUS_EXTENSIONS:
            found.add("dangerous-extension")
    else:
        ext = _final_extension(s)
        if ext in DANGEROUS_EXTENSIONS:
            found.add("dangerous-extension")

    return sorted(found)
```

1.1.c · Email, text, and social media attacks

3 pts · auto-graded on the site

Coding problem 1.1.c: Phishing indicator extractor

Skill: Analyze Risk, Detect Attacks · **Maps to:** 1.1.A, 1.1.C · **Language:** Python 3 (pytest)

Problem

Build the indicator extractor a security-awareness tool would use to screen incoming messages across channels. Given a message dict with channel, sender, text, and link fields, return the sorted list of phishing indicator labels that the message triggers.

A message may trigger zero, one, or several indicators. The result is the sorted, de-duplicated list of triggered label strings.

Function contract

Implement `solve(message: dict) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A dict with the following keys (all strings):

- `"channel"`: one of `"email"`, `"sms"`, `"dm"` (social media direct message)
- `"sender"`: the sender address, phone number, or handle
- `"text"`: the message body text
- `"link"`: a URL or empty string

Output

A **sorted, de-duplicated** `list[str]` drawn from the five indicator labels:

- `"lookalike-domain"`: sender contains a mangled brand domain
- `"urgent-money-request"`: text requests an urgent financial transaction
- `"credential-request"`: text requests credentials or account verification
- `"mismatched-sender"`: text claims a brand the sender address does not belong to
- `"shortened-link"`: link uses a URL shortener or bare IP address

See `interface.md` for the precise indicator rule sets.

Examples

```
{
  "channel": "email",
  "sender": "security@paypa1.com",
  "text": "Verify your PayPal account now.",
  "link": ""
}
-> ["credential-request", "lookalike-domain", "mismatched-sender"]

{
  "channel": "sms",
  "sender": "+1-555-0192",
  "text": "Reminder: dentist at 2pm tomorrow. Reply YES.",
  "link": ""
}
-> []

{
  "channel": "email",
  "sender": "ceo@corp-executive.net",
  "text": "Wire $50k today, do not loop in finance, keep this confidential.",
  "link": ""
}
-> ["urgent-money-request"]

{
  "channel": "dm",
  "sender": "@IT_support99",
  "text": "Your account expires tonight, verify here.",
  "link": "http://192.168.1.1/login"
}
-> ["credential-request", "shortened-link"]

{
  "channel": "sms",
  "sender": "NETFLIX",
  "text": "Your Netflix payment failed. Update your payment at netflix.com/billing",
  "link": "https://bit.ly/3xAmPle"
}
-> ["credential-request", "shortened-link"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
import re
from typing import List


_LOOKALIKE_FRAGMENTS = [
    "paypa1", "pay-pal", "paypall",
    "amaz0n", "amazzon", "amazon-security",
    "g00gle", "go0gle", "googie",
    "micros0ft", "microsooft", "micosoft",
    "app1e", "appie", "app-le",
    "netfl1x", "netf1ix",
    "secure-login", "account-secure", "account-update", "login-secure",
    "verify-account", "update-billing",
]

_MONEY_PHRASES = [
    "wire transfer", "wire $", "send money", "transfer funds", "pay now",
    "gift card", "buy gift cards", "purchase gift cards", "send gift card",
    "urgent payment", "immediate payment", "send funds", "money transfer",
]

_CRED_PHRASES = [
    "verify your account", "confirm your password", "enter your password",
    "reset your password", "update your payment", "enter your credit card",
    "provide your ssn", "confirm your identity", "verify your identity",
    "enter your login", "sign in to verify", "log in to verify",
    "confirm your email", "verify your email", "account verification required",
]

_BRANDS = [
    "paypal", "amazon", "google", "microsoft", "apple", "netflix",
    "facebook", "instagram", "twitter", "linkedin", "irs",
    "social security", "bank of america", "wells fargo", "chase", "citibank",
]

_SHORTENER_HOSTS = [
    "bit.ly", "tinyurl.com", "t.co", "goo.gl", "ow.ly", "is.gd",
    "buff.ly", "short.io", "rb.gy", "cutt.ly", "tiny.cc",
]

_IP_URL_RE = re.compile(r"https?://\d+\.\d+\.\d+\.\d+")


def _check_lookalike(sender_low: str) -> bool:
    return any(frag in sender_low for frag in _LOOKALIKE_FRAGMENTS)


def _check_urgent_money(text_low: str) -> bool:
    return any(phrase in text_low for phrase in _MONEY_PHRASES)


def _check_credential(text_low: str) -> bool:
    return any(phrase in text_low for phrase in _CRED_PHRASES)


def _check_mismatched_sender(sender_low: str, text_low: str) -> bool:
    brands_in_text = [b for b in _BRANDS if b in text_low]
    if not brands_in_text:
```

```
        return False
    return not any(b in sender_low for b in brands_in_text)


def _check_shortened_link(link_low: str) -> bool:
    if not link_low:
        return False
    if any(host in link_low for host in _SHORTENER_HOSTS):
        return True
    if _IP_URL_RE.search(link_low):
        return True
    return False


def solve(message: dict) -> List[str]:
    sender_low = (message.get("sender") or "").lower()
    text_low = (message.get("text") or "").lower()
    link_low = (message.get("link") or "").lower()

    found = set()
    if _check_lookalike(sender_low):
        found.add("lookalike-domain")
    if _check_urgent_money(text_low):
        found.add("urgent-money-request")
    if _check_credential(text_low):
        found.add("credential-request")
    if _check_mismatched_sender(sender_low, text_low):
        found.add("mismatched-sender")
    if _check_shortened_link(link_low):
        found.add("shortened-link")
    return sorted(found)
```